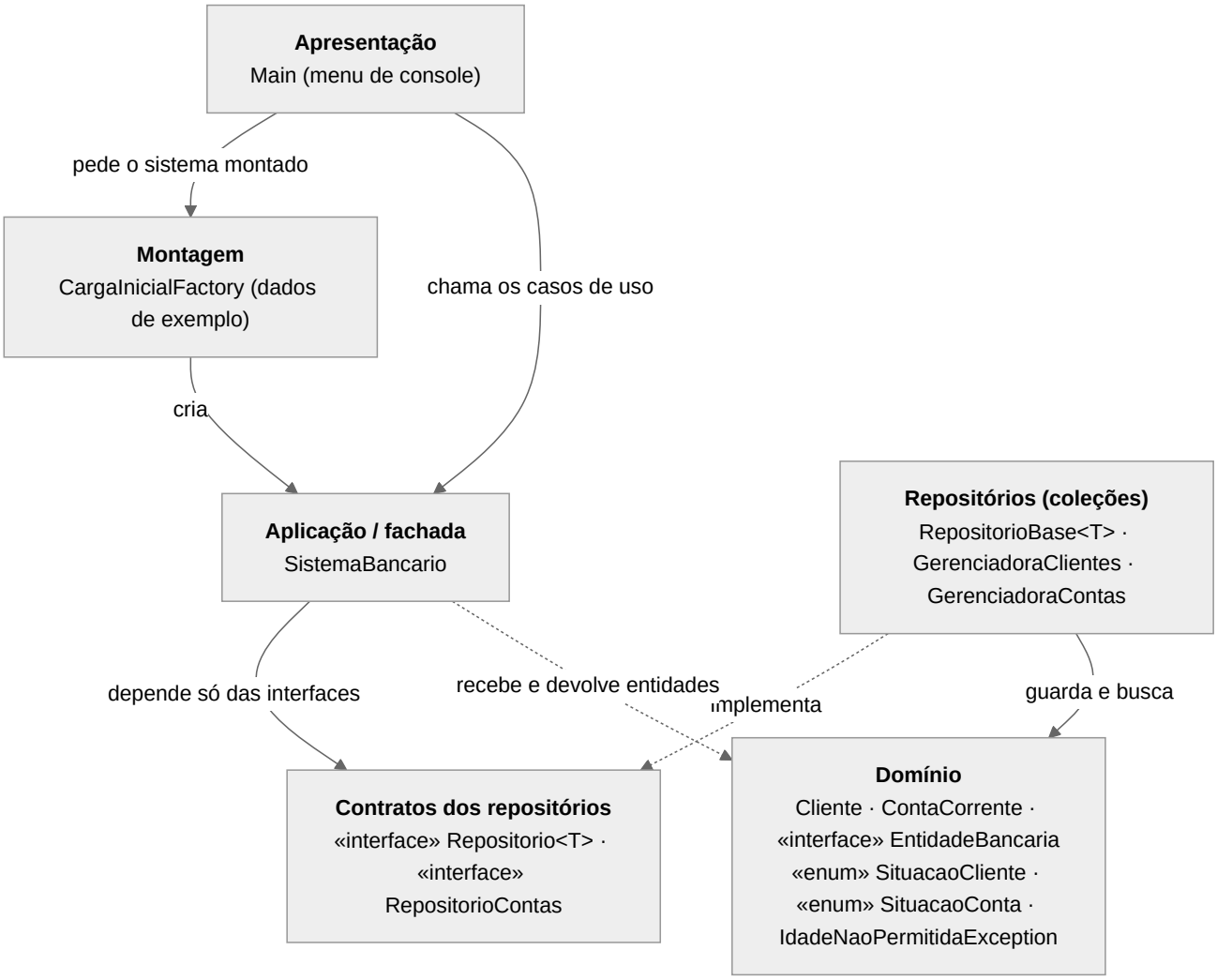
09 — Arquitetura do Sistema Bancário

Este documento descreve a arquitetura do código de produção (`src/`) na versão **final** da entrega, ou seja, depois de TDD, BDD, DDD e das correções pós-auditoria. Onde a estrutura mudou em relação ao código recebido, isso é indicado (a versão original está em `codigo-original/`).

1. Visão geral

É uma aplicação Java de console, sem banco de dados, sem framework e sem interface gráfica. Todos os dados vivem em memória, em listas dentro dos repositórios. Todo o código de produção está em um único pacote, `negocio`, mas as classes se organizam em **camadas lógicas** bem definidas:



A regra de dependência é de **fora para dentro**: `Main` conhece o `SistemaBancario`; o `SistemaBancario` conhece apenas as **interfaces** dos repositórios; os repositórios conhecem as entidades; as entidades não conhecem ninguém além dos seus próprios conceitos (enums e exceção). Nenhuma entidade depende de repositório, fachada ou console.

2. Camadas e responsabilidades

2.1 Apresentação — `Main`

Menu de console com as 12 operações da atividade: clientes (1 a 7), contas correntes (8 a 12) e 0 para sair. Lê cada entrada com `Scanner`, chama o `SistemaBancario` e imprime o resultado. Não contém regra de negócio: se o `SistemaBancario` devolve `null`, `false` ou `Optional` vazio, o `Main` apenas mostra "não encontrado".

Entrada inválida pede o valor de novo, sem derrubar o programa. A opção 3 usa `Cliente.idadePermitida`, a mesma regra do construtor. Execução real: `evidencias/55-menu-console-todas-as-opcoes.txt`.

2.2 Montagem — `CargaInicialFactory`

Static factory method que monta um `SistemaBancario` com dois clientes e duas contas de exemplo, usado só pelo `Main`. Não guarda estado: cada chamada devolve um sistema novo e independente. Os testes **não** usam essa fábrica; cada teste monta seu próprio `SistemaBancario` vazio (ver `10-arquitetura-testes.pdf`).

2.3 Aplicação — `SistemaBancario` (fachada)

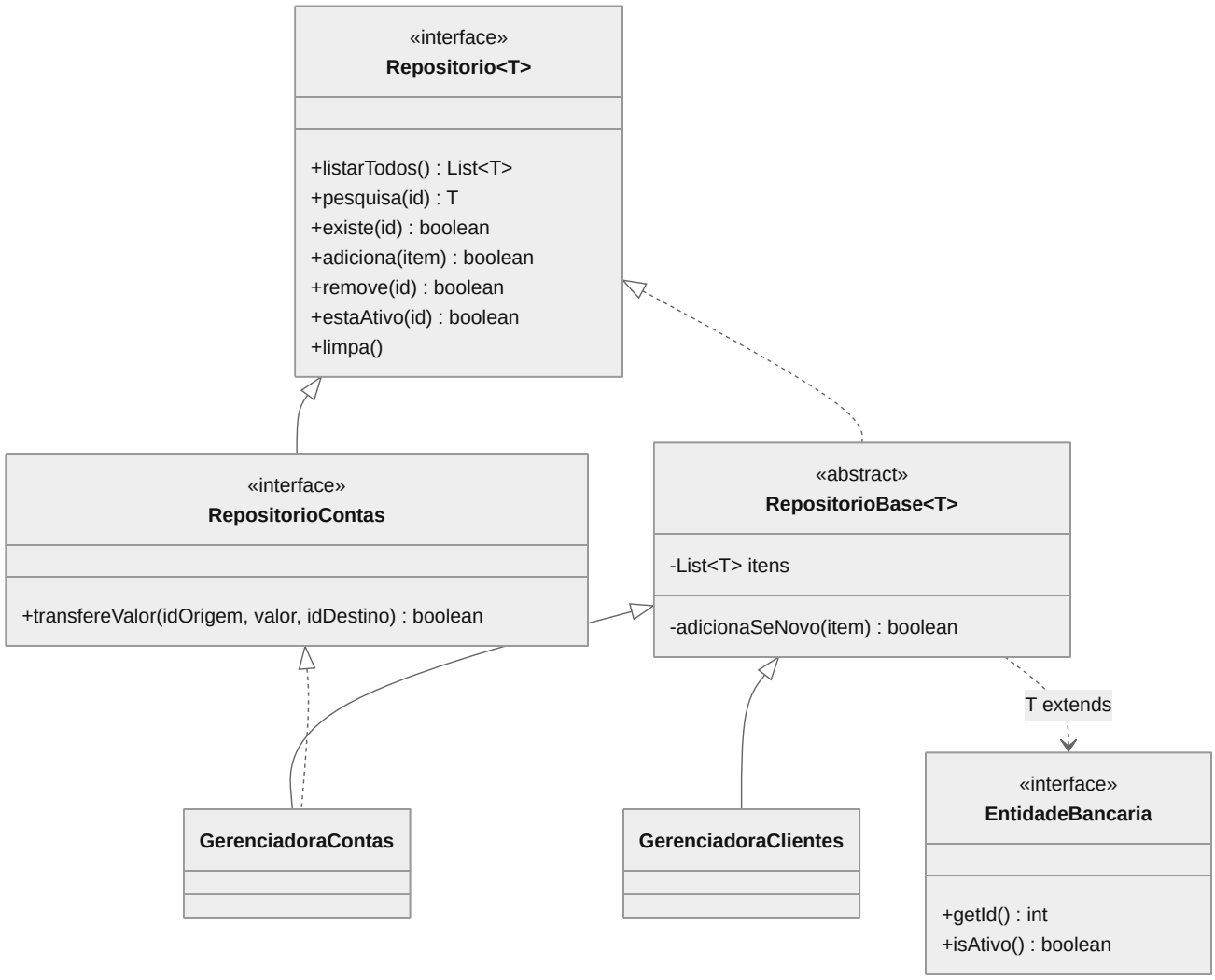
É o ponto de entrada de todos os casos de uso. Recebe os dois repositórios pelo construtor (**injeção de dependência**) e os guarda pelos tipos das interfaces (`Repositorio<Cliente>` e `RepositorioContas`), não pelas classes concretas.

Operação	Requisito	O que faz
`consultarCliente(id)`	1	`gerClientes.pesquisa(id)` → `Cliente` ou `null`
`cadastrarCliente(cliente)`	2	`gerClientes.adiciona(cliente)` → `false` se o id já existe
`ativarCliente(id)`	4	localiza e chama `cliente.ativar()`; `false` se não existe
`desativarCliente(id)`	5	localiza e chama `cliente.desativar()`; `false` se não existe
`verificarSituacaoCliente(id)`	6	`Optional<SituacaoCliente>`; vazio = cliente inexistente
`removerCliente(id)`	7	`gerClientes.remove(id)`
`consultarConta(id)`	8	`gerContas.pesquisa(id)` → `ContaCorrente` ou `null`
`cadastrarConta(conta)`	9	`gerContas.adiciona(conta)` → `false` se o número já existe
`verificarSituacaoConta(id)`	10	`Optional<SituacaoConta>`; vazio = conta inexistente
`removerConta(id)`	11	`gerContas.remove(id)`
`transferir(idOrigem, idDestino, valor)`	12	`gerContas.transfereValor(...)`

O requisito 3 (validar idade) não aparece aqui porque é uma **invariante da entidade** `Cliente`: é verificado no construtor e no `setIdade`, antes de o cliente sequer chegar à fachada.

A fachada é deliberadamente "fina": ela **coordena** (localiza entidades, chama a ação certa, traduz ausência em `false`/`Optional.empty()`), mas não decide regra de negócio. Todas as operações de escrita devolvem `boolean` (sucesso/rejeição), seguindo o padrão que já existia em `ativarCliente` e `remove`.

2.4 Repositórios — coleções de entidades

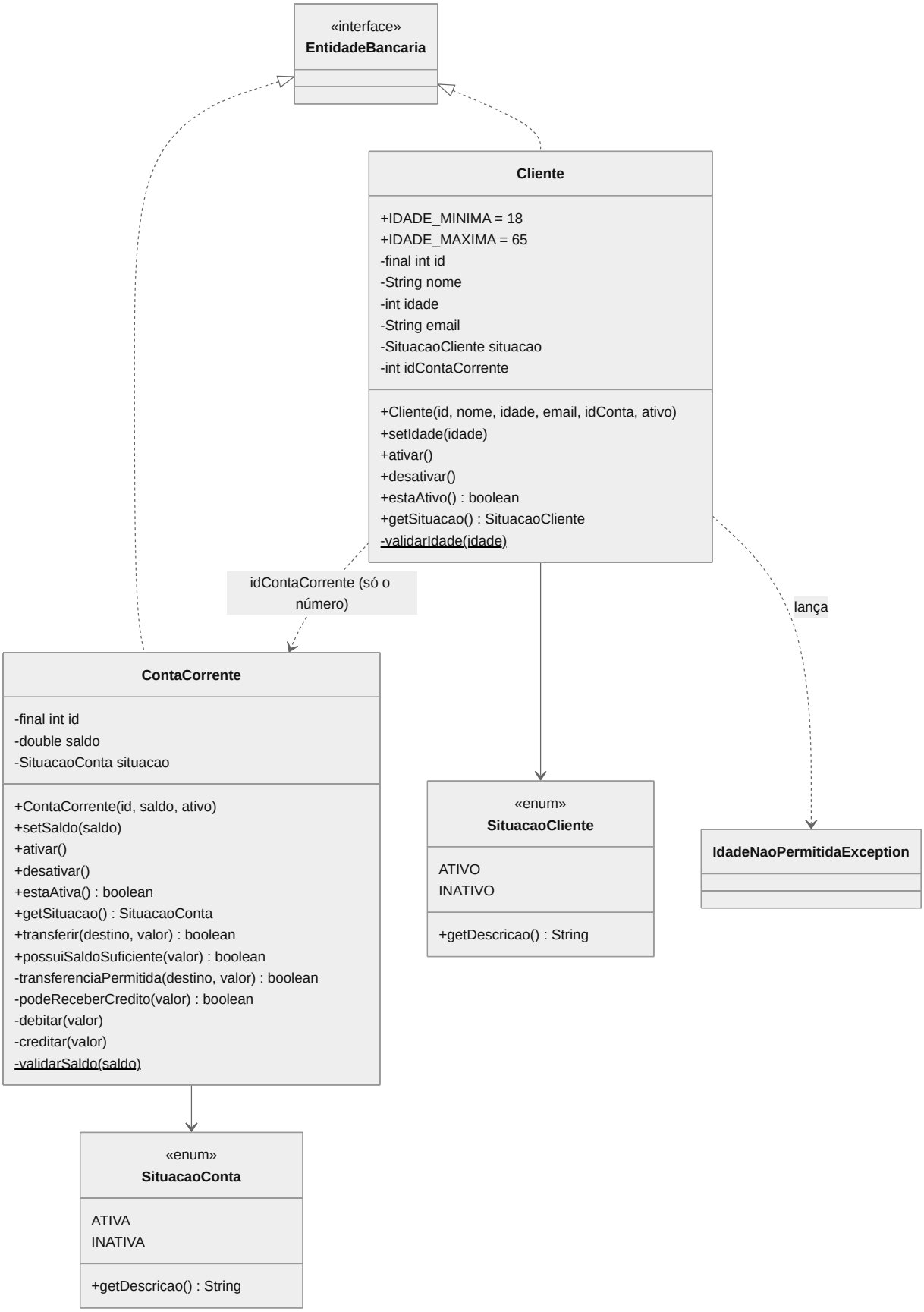


- **`Repositorio<T extends EntidadeBancaria>`**: contrato genérico de CRUD. O limite `T extends EntidadeBancaria` é o que permite ao repositório buscar por `getId()` sem saber se está lidando com cliente ou conta.
- **`RepositorioBase<T>`**: implementação única do CRUD sobre uma `ArrayList`. Foi criada na atividade anterior para eliminar a duplicação que existia entre as duas gerenciadoras. Nesta entrega, após a auditoria, ela passou a ser a **dona da regra de unicidade do id**:
  - a lista recebida no construtor é **copiada** (alterar a lista original depois não afeta o repositório);
  - uma carga inicial com id repetido lança `IllegalArgumentException`;
  - `adiciona` devolve `false` se o id já existe;
  - `listarTodos()` devolve uma lista **somente leitura** (`Collections.unmodifiableList`), então ninguém consegue inserir por fora.

- **`GerenciadoraClientes`**: não acrescenta nada; existe para dar nome de domínio ao repositório de clientes.
- **`GerenciadoraContas`** (implementa `RepositorioContas`): acrescenta `transfereValor`, que apenas **localiza** as duas contas pelo id e delega a regra para `ContaCorrente.transferir`.

Nota de nomenclatura: os comentários do código descrevem `RepositorioBase` como *Template Method*. A rigor, ela é uma **classe base com implementação comum reutilizada por herança**: não há um método-modelo que chame passos abstratos definidos nas subclasses. Vale saber essa distinção se a pergunta surgir na apresentação.

2.5 Domínio — entidades e conceitos



- `Cliente` e `ContaCorrente` são **entidades**: têm identidade (`id`, agora `final`, sem `setId`) e protegem o próprio estado.

- **`SituacaoCliente`** e **`SituacaoConta`** substituíram o `boolean ativo` original. São enums separados porque são conceitos diferentes do negócio, mesmo tendo hoje os mesmos dois valores.
- **`IdadeNaoPermitidaException`** é a exceção checada da regra de idade (já existia no original).
- A ligação cliente → conta é **fraca**: o cliente guarda só o número (`idContaCorrente`), não uma referência ao objeto. Por isso remover um não remove o outro (decisão documentada em `03-ddd.pdf`).

`setAtivo(boolean)`, `isAtivo()` e o parâmetro `boolean ativo` dos construtores foram **mantidos por compatibilidade** com o `TesteUnitarioManual` e a `CargaInicialFactory`; internamente eles chamam `ativar()`/`desativar()`.

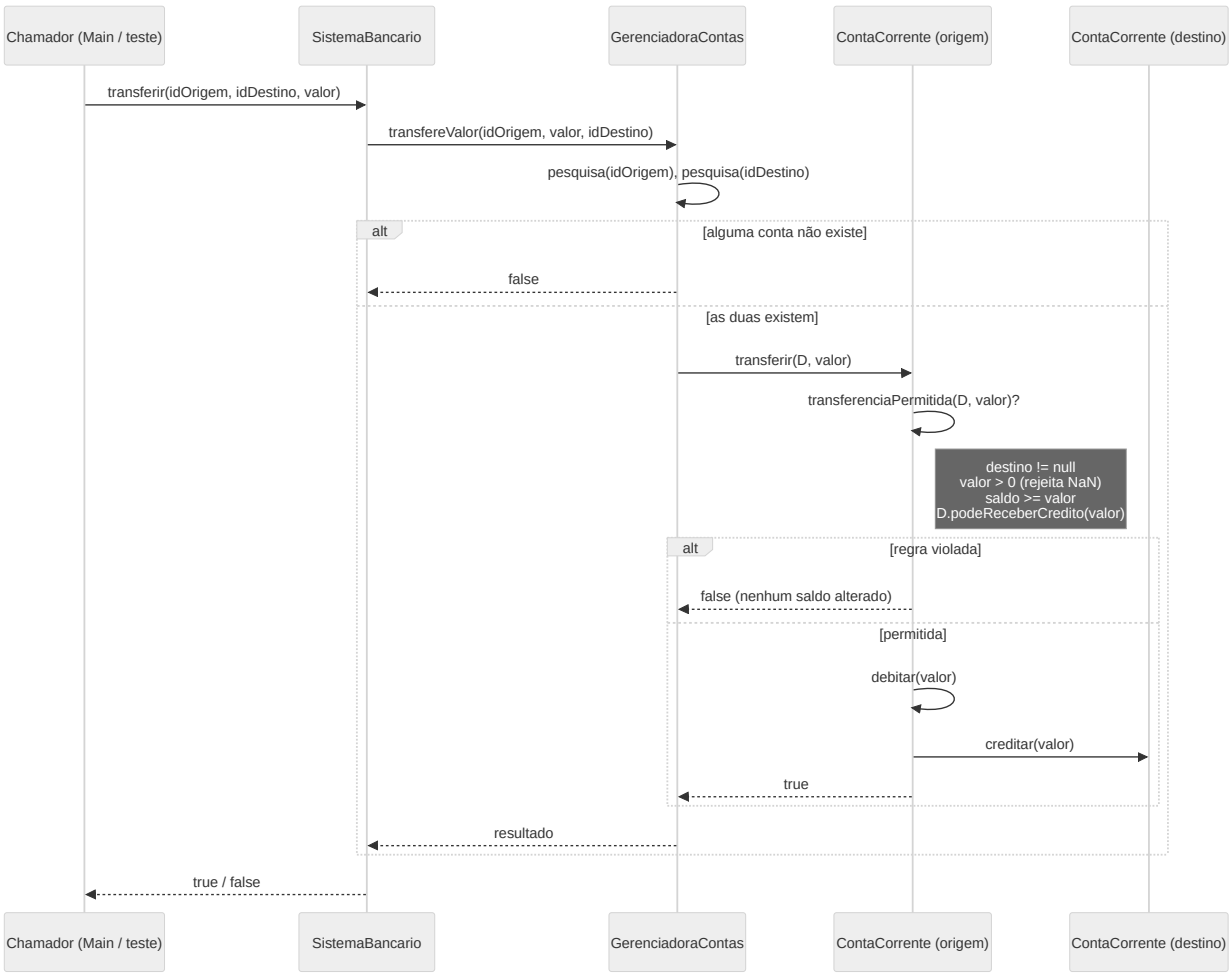
3. Onde mora cada regra de negócio

Regra	Onde é garantida	Tipo
Idade entre 18 e 65 (inclusive), na criação **e** na alteração	`Cliente.validarIdade` (construtor e `setIdade`)	Invariante da entidade
Saldo sempre finito (nunca `Infinity`/`NaN`)	`ContaCorrente.validarSaldo` (construtor e `setSaldo`)	Invariante da entidade
Identidade não muda	`id final` em `Cliente` e `ContaCorrente`	Invariante da entidade
Ativar / desativar, situação	`ativar()`, `desativar()`, `getSituacao()` nas entidades	Comportamento da entidade
Transferência: destino existe, valor > 0, saldo suficiente, destino não estoura	`ContaCorrente.transferenciaPermitida`	Regra da entidade (verificada **antes** de qualquer débito)
Débito e crédito	`ContaCorrente.debitar` / `creditar` (privados)	Encapsulamento da entidade
Id único dentro do banco	`RepositorioBase` (`adicionaSeNovo`, cópia da lista, lista somente leitura)	Regra da **coleção**
Remoção	`RepositorioBase.remove`	Operação sobre a coleção
Localizar contas para transferir	`GerenciadoraContas.transfereValor`	Coordenação
"Inexistente" ≠ "inativo"	`SistemaBancario.verificarSituacao*` (`Optional`)	Caso de uso

O critério usado: se a regra depende só dos dados de **um objeto**, ela fica no objeto; se depende de **todos os objetos** (unicidade), fica em quem guarda a coleção; se só **combina** coisas, fica na fachada.

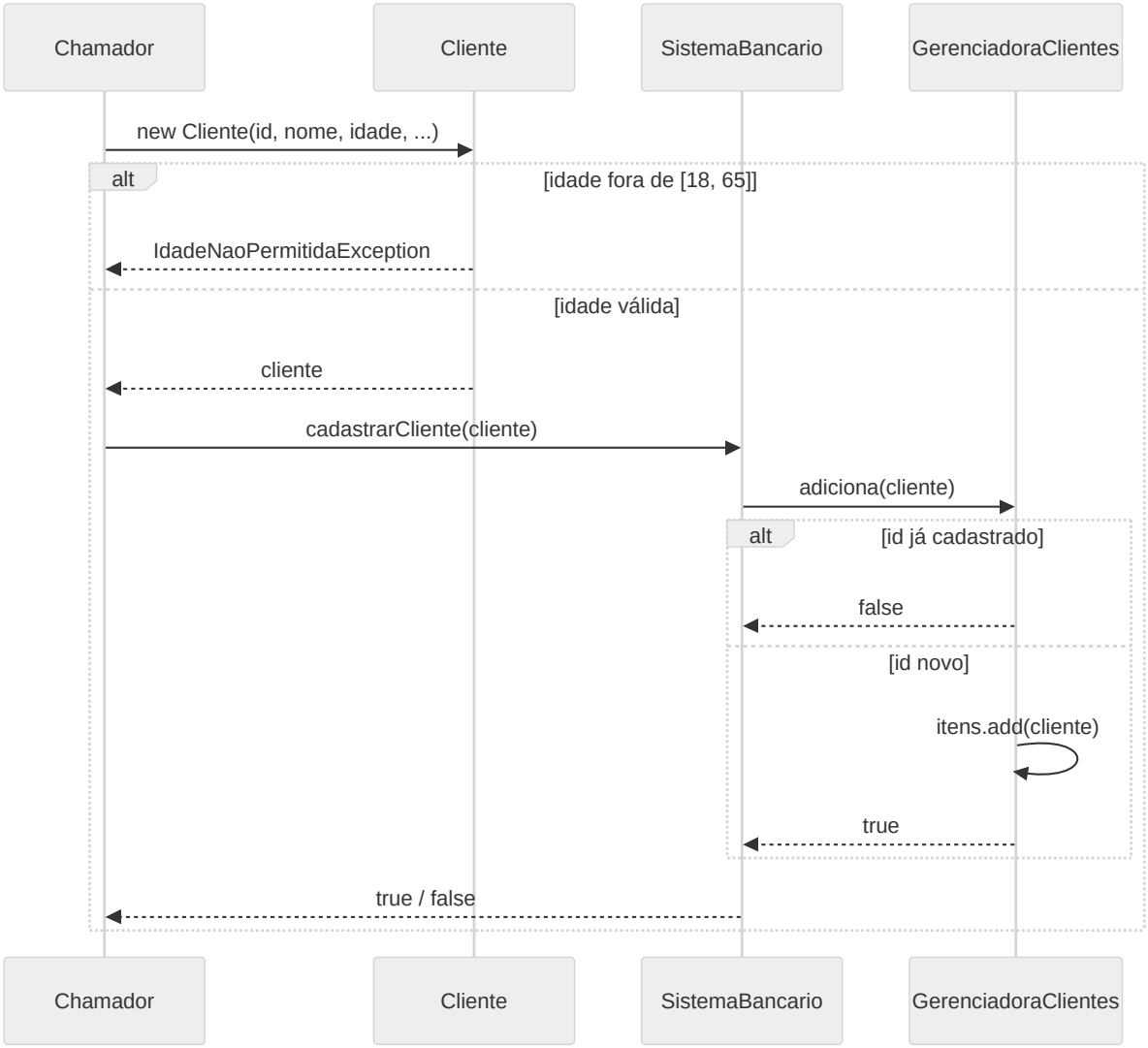
4. Fluxos principais

4.1 Transferência (requisito 12)



No código original, o débito e o crédito eram feitos **dentro do repositório** com `setSaldo(getSaldo() - valor)`, e a única checagem era `saldo < valor`. Isso deixava passar valor negativo (o dinheiro andava no sentido contrário). Agora a conta decide e o repositório só localiza.

4.2 Cadastro de cliente (requisitos 2 e 3)



Um cliente com idade inválida nunca chega a existir, então o repositório nunca precisa se preocupar com idade. O cadastro de conta (requisito 9) segue o mesmo desenho, com a invariante de saldo finito no lugar da idade.

4.3 Verificar situação (requisitos 6 e 10)

```
SistemaBancario.verificarSituacaoCliente(id)
    └─ Optional.ofNullable(gerClientes.pesquisa(id))   → vazio se não existe
          .map(Cliente::getSituacao)                   → ATIVO / INATIVO
```

5. Padrões e princípios aplicados

Padrão / princípio	Onde	Para quê
Fachada	`SistemaBancario`	Um único ponto de entrada para os casos de uso

Padrão / princípio	Onde	Para quê
Repositório (DDD)	`Repositorio`, `RepositorioBase`, `Gerenciadora*`	Isolar a coleção das entidades
Injeção de dependência por construtor	`SistemaBancario(Repositorio<Cliente>, RepositorioContas)`	Montar o sistema com qualquer implementação (inclusive nos testes)
Inversão de dependência	fachada depende de interfaces	Trocar a implementação sem alterar a fachada
Static factory method	`CargaInicialFactory.criarComDadosDeExemplo()`	Separar dados de exemplo do domínio
Entidade e **objeto de valor** (DDD)	`Cliente`/`ContaCorrente`; `SituacaoCliente`/`SituacaoConta`	Identidade vs. valor imutável
Tell, don't ask	`origem.transferir(destino, valor)` em vez de `setSaldo(getSaldo() - v)`	A regra fica com quem tem os dados
Fail fast	construtores de `Cliente` e `ContaCorrente`	Nenhum objeto nasce inválido

6. Estrutura de pastas e build

```
SistemaBancario/
├── pom.xml, mvnw, mvnw.cmd, .mvn/   build Maven (adicionado)
├── src/                             código de produção  (sourceDirectory = src)
│   ├── negocio/                     todas as classes descritas acima
│   └── teste/TesteUnitarioManual    teste manual original (é compilado junto com src)
├── test/java/{tdd,bdd,ddd}          testes automatizados (testSourceDirectory)
├── test/resources/features          cenários .feature do Cucumber (testResources)
└── docs/                            documentação e evidências
```

O projeto **não** segue o layout padrão do Maven (`src/main/java`, `src/test/java`). O `pom.xml` aponta explicitamente `sourceDirectory=src` e `testSourceDirectory=test/java` para não quebrar o projeto Eclipse recebido. Consequência prática: o `TesteUnitarioManual`, que está em `src/teste`, é compilado como código de produção e roda com `java -cp target/classes teste.TesteUnitarioManual`, fora do Surefire.

Tecnologias: Java 8 (`maven.compiler.release=8`), JUnit 4.13.2, Cucumber 7.20.1 (só em escopo `test`). Nenhuma dependência em tempo de execução.

7. Comparação com o código recebido

Aspecto	Original	Final
Situação	`boolean ativo` + `setAtivo`	enums `SituacaoCliente`/`SituacaoConta` + `ativar()`/`desativar()`
Identidade	`setId` público	`id final`
Idade	validada só no construtor	construtor **e** `setIdade`
Saldo	qualquer `double`	sempre finito

Aspecto	Original	Final
Transferência	regra no repositório, aceitava valor ≤ 0	regra em `ContaCorrente`, todas as condições antes do débito
Unicidade de id	inexistente	garantida pelo repositório
Fachada	4 operações	11 operações (todos os requisitos exceto o 3, que é invariante)
Build / testes	projeto Eclipse, JUnit declarado mas não usado	Maven Wrapper, JUnit 4 + Cucumber, 55 testes automatizados

Diferença completa: `diff-original-para-final.diff`.

8. Limitações conhecidas

- O `Main` cobre as 12 operações, mas não tem opção para ativar/desativar conta.
- Valores monetários em `double` (sem exatidão decimal); `BigDecimal` fica como recomendação.
- `RepositorioBase.estaAtivo(id)` devolve `false` para inativo **e** inexistente (mantido pelo `TesteUnitarioManual`). O caminho correto é `SistemaBancario.verificarSituacao*`.
- Não há regra que impeça operações de clientes ou contas inativos (ex.: transferir de conta inativa), porque o projeto não define essa regra. A discussão está em `03-ddd.pdf`.
- Transferência para a própria conta não é bloqueada (saldo final igual).